SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	Batch 47-B
Date and Day of Assignment	**Week6 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Name	A.Pushpanjali	**Hall Ticket**	2303A54050
Assignment Number: 11.1(Present assignment number)/**24**(Total number of assignments)			

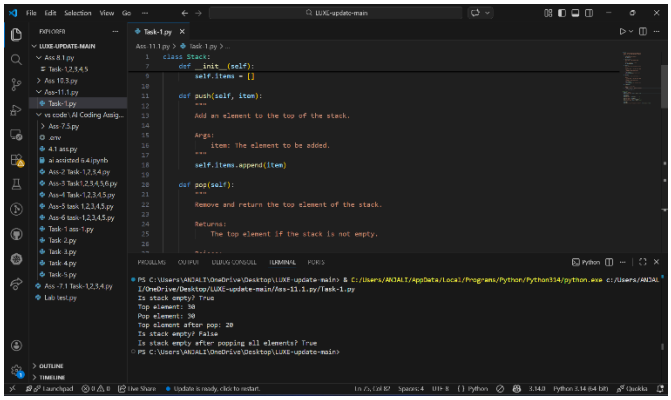
Q.No.	Question	*Expected Time to complete*
1	**Lab 11 – Data Structures with AI: Implementing Fundamental Structures** **Lab Objectives** • Use AI to assist in designing and implementing fundamental data	Week6 - Monday

structures in Python.

- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.



Sample Input Code:

s = Stack()
s.push(10)
s.push(20)
s.push(30)
print(s.peek())
print(s.pop())
print(s.is_empty())

Explanation:

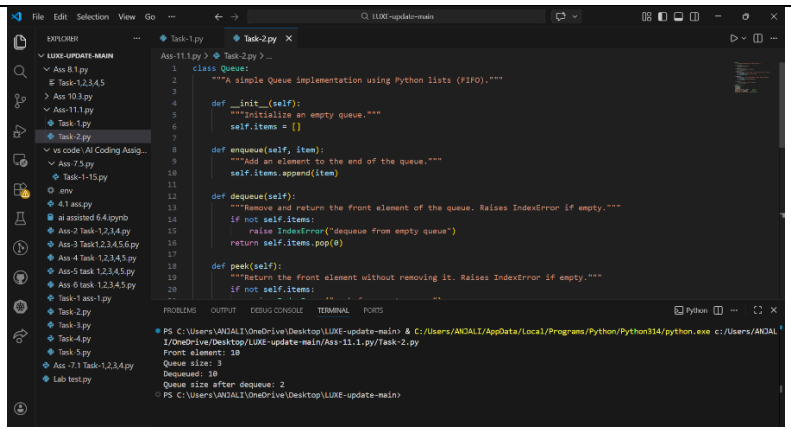
A **Stack** follows the **Last In First Out (LIFO)** principle. The last element pushed into the stack will be the first element removed.

Expected Output:

30
30
False

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.



Sample Input Code:

```
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
print(q.peek())
print(q.dequeue())
print(q.size())
```

Explanation:

A **Queue** follows the **First In First Out (FIFO)** principle. The first element inserted will be the first element removed.

Expected Output:

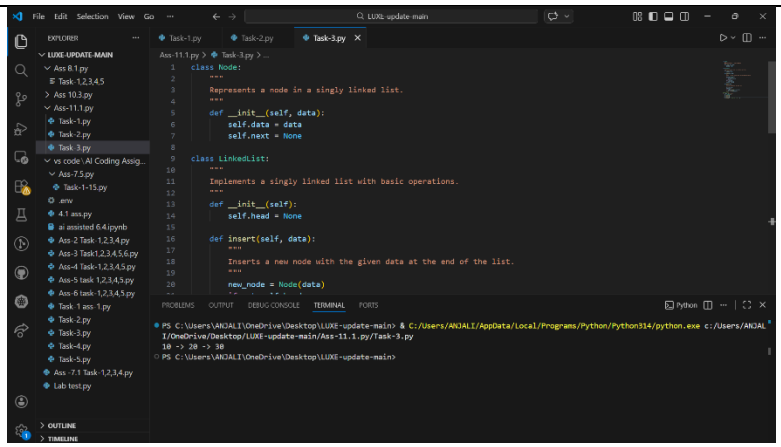
```
10
10
2
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Explanation:

A **Singly Linked List** stores elements in nodes where each node points to the next node. It allows dynamic insertion of elements.



Sample Input Code:

ll = LinkedList()

ll.insert(10)

ll.insert(20)

ll.insert(30)

ll.display()

Expected Output:

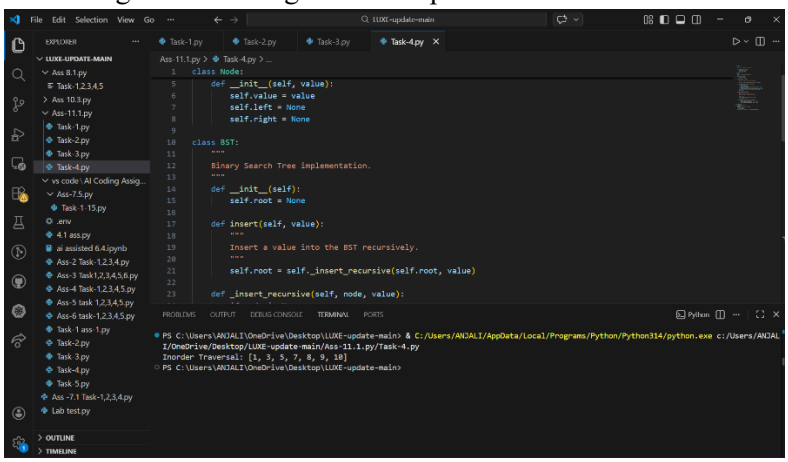
10 -> 20 -> 30

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Explanation:

A **Binary Search Tree** stores elements such that the left child is smaller and the right child is larger than the parent node.



Sample Input Code:

bst = BST()

bst.insert(50)

bst.insert(30)

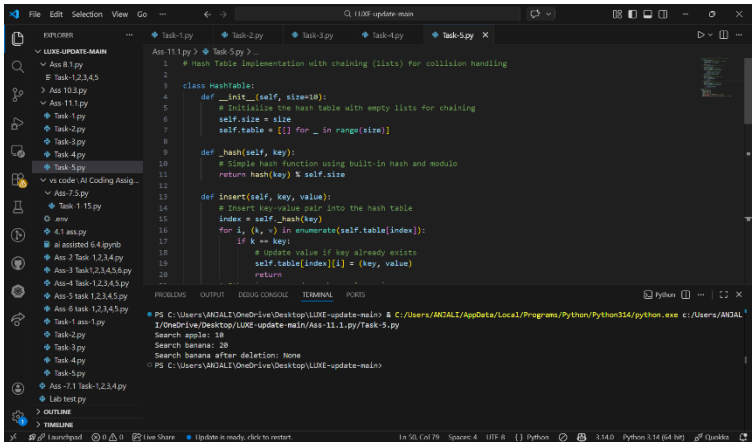
bst.insert(70)

bst.insert(20)

bst.insert(40)

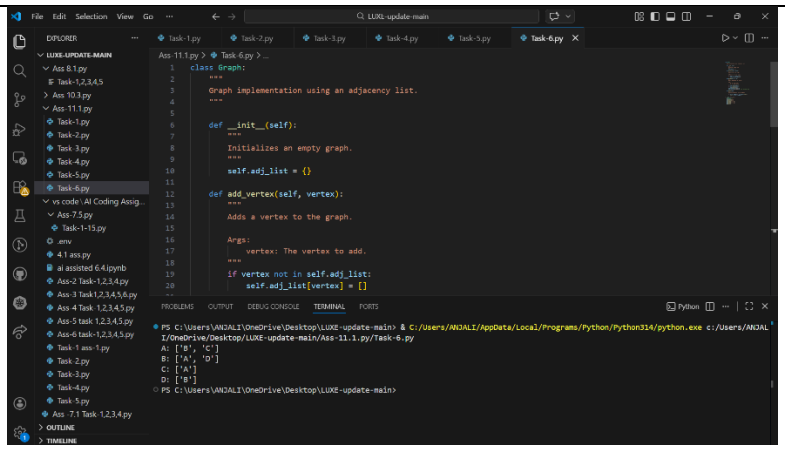
bst.inorder()
Expected Output:
20 30 40 50 70

Task Description #5 – Hash Table
Task: Use AI to implement a hash table with basic insert, search, and delete methods.
Explanation:
A **Hash Table** stores key–value pairs and allows fast searching using a hash function. Collisions are handled using chaining



Sample Input Code:
h = HashTable()
h.insert("101", "Alice")
h.insert("102", "Bob")
print(h.search("101"))
h.delete("102")
print(h.search("102"))
Expected Output:
Alice
None

Task Description #6 – Graph Representation
Task: Use AI to implement a graph using an adjacency list.
Explanation:
A **Graph** represents connections between nodes. An **adjacency list** stores each node and its connected neighbors.



Sample Input Code:

g = Graph()

g.add_vertex("A")

g.add_vertex("B")

g.add_edge("A", "B")

g.display()

Expected Output:

A: ['B', 'C']

B: ['A', 'D']

C: ['A']

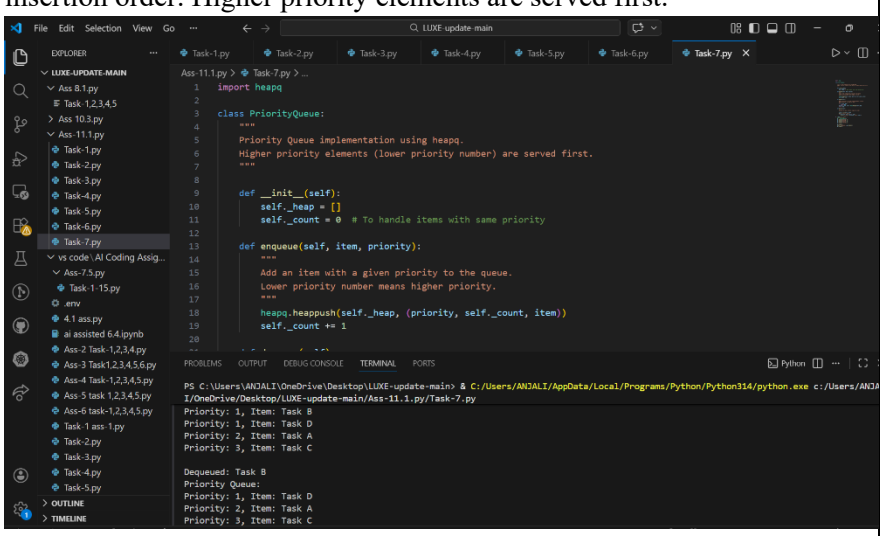
D: ['B']

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Explanation:

A **Priority Queue** processes elements based on priority instead of insertion order. Higher priority elements are served first.



Sample Input Code:

```
pq = PriorityQueue()
pq.enqueue("Task1", 2)
pq.enqueue("Task2", 1)
pq.enqueue("Task3", 3)
print(pq.dequeue())
```

Expected Output:

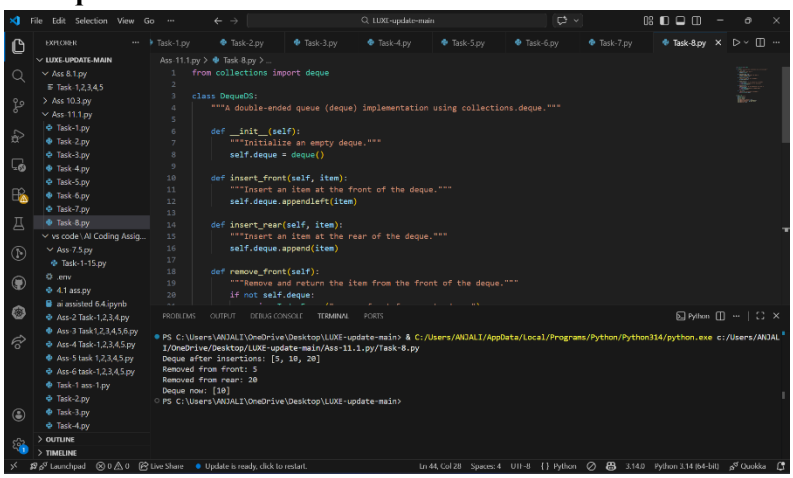
Task2

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Explanation:

A **Deque** allows insertion and deletion from both the front and rear.



Sample Input Code:

```
d = DequeDS()
d.insert_front(10)
d.insert_rear(20)
d.insert_front(5)
d.remove_rear()
print(d.display())
```

Expected Output:

[5, 10]

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with

quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

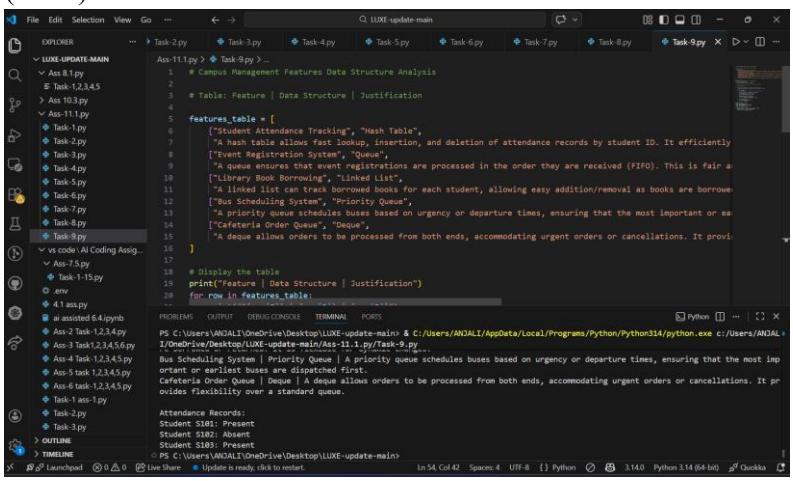
- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

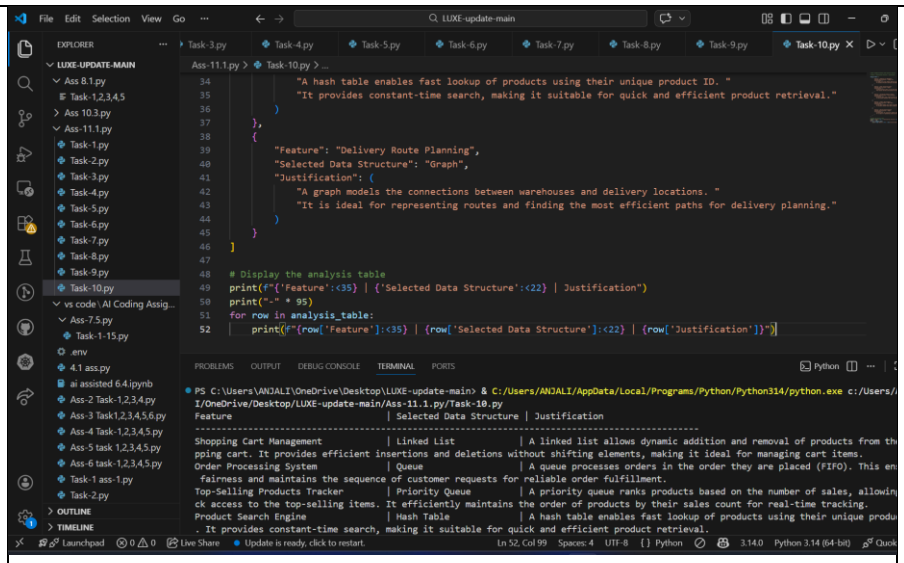
- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Explanation:

Different data structures are chosen based on how each feature handles data. **Deque** is used for student attendance because students enter and exit from both ends. **Hash Table** helps manage event registrations with fast search using IDs. **Stack** can manage library books using the LIFO principle. **Graph** represents bus routes as connected stops. **Queue** is used for cafeteria orders since students are served in the order they arrive (FIFO)



	Expected Output: Attendance Records: Student S101: Present Student S102: Absent Student S103: Present **Task Description #10: Smart E-Commerce Platform – Data Structure Challenge** An e-commerce company wants to build a Smart Online Shopping System with: 1. Shopping Cart Management – Add and remove products dynamically. 2. Order Processing System – Orders processed in the order they are placed. 3. Top-Selling Products Tracker – Products ranked by sales count. 4. Product Search Engine – Fast lookup of products using product ID. 5. Delivery Route Planning – Connect warehouses and delivery locations. **Student Task:** • For each feature, select the most appropriate data structure from the list below: o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque • Justify your choice in 2–3 sentences per feature.	



- Implement one selected feature as a working Python program with

Explanation:

Different data structures improve efficiency in an online shopping system. **Linked List** allows easy addition and removal of products in a shopping cart. **Queue** processes orders in the order they are placed. **Priority Queue** tracks top-selling products based on sales priority. **Hash Table** enables fast product search using product IDs. **Graph** helps plan delivery routes between warehouses and locations.

Expected Output:

Feature	**Example Input**	
Shopping Cart Management	Add: Phone, Laptop	Cart → P
Order Processing System	Order1, Order2	Processin
Top-Selling Products Tracker	ProductA(10), ProductB(20)	Top Prod
Product Search Engine	Search ID101	Product F
Delivery Route Planning	Warehouse → CityA	Route: W